

Prague University of Economics and Business  
Faculty of Informatics and Statistics



# **A Data Integration Framework for Enterprise Application Security**

## **MASTER'S THESIS**

Study program: Applied Data Analytics and Artificial Intelligence

Specialization: Retail and E-Commerce Data Analytics

Author: Bc. Vojtěch Kraus

Supervisor: Ing. Aleš Kotuč

Prague, month YYYY

## **Acknowledgements**

I would like to thank Ing. Aleš Kotuč for his guidance and valuable feedback.

## **Abstract**

To protect their business-critical applications from cybersecurity threats, enterprise security teams rely on numerous tools to detect and resolve vulnerabilities in application source code, configuration, dependencies, CI/CD pipelines, and runtime infrastructure. These tools use different integration patterns, protocols, and data formats, making it difficult to consistently parse, deduplicate, triage, and group their findings. In large environments, additional challenges emerge when linking all findings to the corresponding business applications. This thesis presents a data framework to tackle those challenges, delivering three distinct contributions: (1) a requirements specification for an application security data platform, (2) a reusable and extensible design covering architecture, data model, and medallion-based data pipeline, and (3) a reference implementation built on the Databricks platform. Together, these contributions offer application security teams a foundation for building a robust data platform independent of individual cybersecurity vendors, relieving them of risks usually associated with vendor lock-in.

## **Keywords**

application security, software security, DevSecOps, security integration, data consolidation, medallion architecture, Databricks

# Contents

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>Introduction</b>                                        | <b>11</b> |
| Motivation . . . . .                                       | 11        |
| Objective . . . . .                                        | 11        |
| Methodology . . . . .                                      | 11        |
| Structure . . . . .                                        | 11        |
| <b>1 Analysis</b>                                          | <b>12</b> |
| 1.1 Asset Discovery and Inventory . . . . .                | 12        |
| 1.1.1 Application Portfolio Management . . . . .           | 12        |
| 1.1.2 Configuration Management Databases . . . . .         | 13        |
| 1.1.3 Integration Options . . . . .                        | 14        |
| 1.1.4 Cloud Asset Discovery . . . . .                      | 15        |
| 1.1.5 Cross-Layer Correlation . . . . .                    | 16        |
| 1.2 Software Development . . . . .                         | 16        |
| 1.2.1 Source Code Management . . . . .                     | 17        |
| 1.2.2 Continuous Integration and Delivery . . . . .        | 17        |
| 1.2.3 Issue Tracking . . . . .                             | 18        |
| 1.3 Static Application Security . . . . .                  | 19        |
| 1.3.1 Static Application Security Testing . . . . .        | 20        |
| 1.3.2 Software Composition Analysis . . . . .              | 21        |
| 1.3.3 Secret Detection . . . . .                           | 21        |
| 1.3.4 Container Image Scanning . . . . .                   | 22        |
| 1.3.5 Infrastructure as Code Security . . . . .            | 23        |
| 1.4 Dynamic Application Security . . . . .                 | 23        |
| 1.4.1 Dynamic Application Security Testing . . . . .       | 23        |
| 1.4.2 Penetration Testing . . . . .                        | 24        |
| 1.4.3 Web Application Firewalls . . . . .                  | 24        |
| 1.4.4 Runtime Application Self-Protection . . . . .        | 25        |
| 1.4.5 Runtime API Security . . . . .                       | 25        |
| 1.4.6 Vulnerability Management and Cloud Posture . . . . . | 26        |
| 1.4.7 Source Integration Summary . . . . .                 | 26        |
| 1.5 Data Engineering . . . . .                             | 27        |
| 1.5.1 Data Platform Architecture . . . . .                 | 27        |
| 1.5.2 Data Integration Patterns . . . . .                  | 28        |
| 1.5.3 Domain Data Model . . . . .                          | 29        |
| 1.6 Related Work and Gap Analysis . . . . .                | 31        |
| 1.6.1 Existing Approaches . . . . .                        | 32        |
| 1.6.2 Databricks Lakewatch . . . . .                       | 32        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 1.6.3    | Comparative Analysis and Research Gap . . . . . | 33        |
| <b>2</b> | <b>Framework</b>                                | <b>35</b> |
| 2.1      | Solution Architecture . . . . .                 | 35        |
| 2.1.1    | Technology Stack . . . . .                      | 35        |
| 2.1.2    | Component Design . . . . .                      | 37        |
| 2.1.3    | Medallion Architecture . . . . .                | 38        |
| 2.2      | Data Model . . . . .                            | 41        |
| 2.2.1    | Schema Patterns . . . . .                       | 41        |
| 2.2.2    | Entity Model . . . . .                          | 43        |
| 2.2.3    | Aggregation Model . . . . .                     | 45        |
| 2.3      | Connector Framework . . . . .                   | 47        |
| 2.3.1    | Connector Abstraction . . . . .                 | 47        |
| 2.3.2    | Ingestion Patterns . . . . .                    | 47        |
| 2.3.3    | Transformation Patterns . . . . .               | 47        |
| 2.4      | Analytics and Serving Layer . . . . .           | 47        |
| 2.4.1    | Aggregation Patterns . . . . .                  | 47        |
| 2.4.2    | ML Workflows . . . . .                          | 47        |
| 2.4.3    | Serving Patterns . . . . .                      | 47        |
| 2.5      | Deployment and Engineering Practices . . . . .  | 47        |
| 2.5.1    | Deployment Strategy . . . . .                   | 47        |
| 2.5.2    | Project Structure . . . . .                     | 47        |
| 2.5.3    | Testing Strategy . . . . .                      | 47        |
| <b>3</b> | <b>Implementation</b>                           | <b>48</b> |
| 3.1      | Environment and Deployment . . . . .            | 49        |
| 3.1.1    | Workspace Setup . . . . .                       | 49        |
| 3.1.2    | Silver Schema . . . . .                         | 49        |
| 3.1.3    | Project Structure and CI/CD . . . . .           | 49        |
| 3.2      | Connectors . . . . .                            | 49        |
| 3.2.1    | ServiceNow . . . . .                            | 49        |
| 3.2.2    | GitHub . . . . .                                | 49        |
| 3.2.3    | GitLab . . . . .                                | 49        |
| 3.2.4    | SonarQube . . . . .                             | 49        |
| 3.2.5    | Semgrep . . . . .                               | 49        |
| 3.2.6    | Dependency-Track . . . . .                      | 49        |
| 3.2.7    | TruffleHog . . . . .                            | 49        |
| 3.2.8    | Vulnerability Enrichment . . . . .              | 49        |
| 3.3      | Analytics and Serving . . . . .                 | 49        |
| 3.3.1    | Application-Repository Mapping . . . . .        | 49        |
| 3.3.2    | Application Risk Scoring . . . . .              | 49        |
| 3.3.3    | Remediation and Compliance Metrics . . . . .    | 49        |
| 3.3.4    | Vulnerability Trends . . . . .                  | 49        |
| 3.3.5    | Risk Prediction Model . . . . .                 | 49        |

|                   |                                             |           |
|-------------------|---------------------------------------------|-----------|
| 3.3.6             | Serving Layer . . . . .                     | 49        |
| 3.4               | Testing and Validation . . . . .            | 49        |
| <b>Conclusion</b> |                                             | <b>50</b> |
|                   | Thesis Outcomes and Contributions . . . . . | 50        |
|                   | Limitations . . . . .                       | 50        |
|                   | Future Work . . . . .                       | 50        |
| <b>References</b> |                                             | <b>51</b> |
| <b>A</b>          | <b>Generative AI Use Disclosure</b>         | <b>56</b> |
| A.1               | Text Content . . . . .                      | 56        |
| A.2               | Diagrams and Figures . . . . .              | 56        |
| A.3               | Source Code . . . . .                       | 56        |
| A.4               | Requirements Specification . . . . .        | 56        |

# List of Figures

|     |                                   |    |
|-----|-----------------------------------|----|
| 1.1 | Conceptual Domain Model . . . . . | 30 |
| 2.1 | Component Design . . . . .        | 37 |
| 2.2 | Medation Layer Design . . . . .   | 39 |

# List of Tables

|     |                                         |    |
|-----|-----------------------------------------|----|
| 1.1 | Data Source Integration Characteristics | 26 |
| 1.2 | Comparison of Existing Approaches       | 33 |
| 2.1 | Silver Entity Tables                    | 44 |
| 2.2 | Silver Finding Tables                   | 44 |



# List of source codes



# Introduction

**Motivation**

**Objective**

**Methodology**

**Structure**

# 1. Analysis

This chapter surveys the literature and analyzes the functional domains relevant to building an application security data platform. Each section covers one domain, combining theoretical foundations with practical tool and [Application Programming Interface \(API\)](#) analysis. The focus is on reviewing existing knowledge, observing source system interfaces, data models, and integration patterns, not on prescribing a solution. Technology choices for the target platform are deferred to [Chapter 2](#) and [Chapter 3](#).

[Section 1.1](#) establishes the business and infrastructure asset context underlying all security findings. [Section 1.2](#) examines software development platforms as data sources. Application security tooling is split into two sections: [Section 1.3](#) covers tools that analyze source code and build artifacts before deployment, while [Section 1.4](#) covers tools that test running applications and monitor deployed infrastructure. This separation reflects a fundamental difference in how findings are produced and correlated: static tools identify targets by repository and file path, while dynamic tools identify targets by [Uniform Resource Locator \(URL\)](#), host, or cloud resource, requiring different integration strategies. [Section 1.5](#) covers the data engineering patterns that underpin the framework's design. Finally, [Section 1.6](#) surveys existing solutions and identifies the gap this thesis addresses.

## 1.1 Asset Discovery and Inventory

Asset discovery and inventory is the foundational data source for any application security data platform. It establishes the business context for all security findings. On the business side, organizations maintain application inventories in a [Configuration Management Database \(CMDB\)](#) or a custom-built catalog. On the infrastructure side, cloud providers and container orchestrators expose asset inventories through [APIs](#). Without a reliable catalog of business applications and their mapping to technical and infrastructure assets, a vulnerability discovered in a repository or a running service cannot be attributed to an owner, assessed for business impact, or prioritized against organizational risk criteria.

### 1.1.1 Application Portfolio Management

Enterprise organizations maintain catalogs of their business applications with metadata such as application name, description, ownership information, lifecycle status, and relationships to other assets (AXELOS, 2019).

These registries also capture security-relevant attributes. Information security is commonly assessed along three dimensions known as the [Confidentiality, Integrity, and Availability](#)

(CIA) triad: confidentiality (preventing unauthorized disclosure), integrity (preventing unauthorized modification), and availability (ensuring authorized access) (Stallings & Brown, 2024). Business impact assessments evaluate all three CIA dimensions to produce criticality ratings, commonly structured as tiers: Tier 1 for mission-critical applications, Tier 2 for important supporting systems, and Tier 3 for non-critical internal tools. Separately, regulatory scope flags identify which compliance frameworks apply to a given application, further contributing to the security requirement set.

The most important attribute from a data integration perspective is the mapping between business applications and their technical assets. A single business application may span multiple source code repositories, microservices, infrastructure components, and deployment environments. This mapping serves as the glue connecting technical security findings to business context. When the framework ingests a vulnerability from a [Static Application Security Testing \(SAST\)](#) scanner linked to a specific repository, the application inventory mapping determines which business application is affected, who owns it, and how critical it is. This enables the risk-aware prioritization that stakeholders require.

### 1.1.2 Configuration Management Databases

The most common [CMDB](#) platform in enterprise environments is ServiceNow, holding over 44% of the global [IT Service Management \(ITSM\)](#) market share (Gartner, 2024). It serves as the reference platform for this thesis. A [CMDB](#) organizes all managed assets as [Configuration Items \(CIs\)](#), each with attributes describing its type, status, and ownership (AXELOS, 2019).

ServiceNow arranges [CIs](#) in a class hierarchy rooted at the `cmdb_ci` base table. Each subclass inherits base attributes (name, `sys_id`, operational status, environment) and adds domain-specific fields. For application inventory purposes, the most relevant class is `cmdb_ci_app1` (Application), which extends the base [CI](#) with attributes such as version, business unit, used-for classification, and references to the owning support group (ServiceNow, 2024a).

Beyond the application record itself, the [CMDB](#) captures relationships between [CIs](#) through a dedicated relationship table (`cmdb_rel_ci`). These relationships model dependencies such as “runs on” (application to server), “depends on” (application to database), and “hosted on” (application to cloud service). For the framework, the most valuable relationships are those linking business applications to technical assets that correspond to entities in the security data: repository names, deployment targets, and infrastructure components. ServiceNow also allows organizations to define custom attributes and relationship types, so the specific fields available for ingestion may vary between deployments.

### 1.1.3 Integration Options

Integrating [CMDB](#) data into an external data platform requires choosing between two strategies: pulling data from ServiceNow via its [APIs](#), or pushing data from a custom ServiceNow application to an external target.

#### ServiceNow APIs

ServiceNow exposes two complementary [Representational State Transfer \(REST\) APIs](#) for [CMDB](#) data extraction. The [Table API](#) provides generic CRUD operations against any ServiceNow table through the endpoint `/api/now/table/{tableName}` (ServiceNow, 2024c). Responses are [JavaScript Object Notation \(JSON\)](#)-formatted, with support for server-side filtering (`sysparm_query`), field selection (`sysparm_fields`), and pagination via `sysparm_limit` and `sysparm_offset` (default maximum of 10 000 records per request). The `sysparm_display_value` parameter resolves reference fields to human-readable names instead of internal `sys_id` values.

The [CMDB Instance API](#) (`/now/cmdb/instance/{className}`) provides a [CMDB](#)-aware alternative that understands the [CI](#) class hierarchy (ServiceNow, 2024a). It can retrieve a [CI](#) with its relationships in a single call, reducing the number of [API](#) calls needed to reconstruct the application-to-asset mapping. However, it is more constrained in its query capabilities than the [Table API](#).

Both [APIs](#) require authentication, typically through Basic Authentication with a service account granted the `rest_service` or `cmdb_read` role, or through OAuth 2.0 for more security-conscious deployments. Rate limits vary by instance configuration and are governed by platform transaction quotas rather than per-endpoint limits. For incremental data extraction, the `sys_updated_on` timestamp field present on all records serves as a reliable high-water mark, allowing consumers to query only records modified since the last extraction.

Any pull-based integration must handle authentication, pagination, rate limiting, and incremental state management. Several integration libraries and platform-native connectors abstract these concerns; the specific tooling choice depends on the target data platform and is discussed in [Section 3.2.1](#).

#### ServiceNow Application

An alternative to pulling data is to push it from the source. ServiceNow supports building scoped applications that run on the platform itself. A custom application can use Outbound REST Messages to send [CMDB](#) data to an external target (e.g., a cloud storage bucket or a [REST](#) endpoint) on a schedule defined through Scheduled Jobs or Flow Designer. The

application can also expose a Scripted [REST API](#) (ServiceNow, 2024b), a custom endpoint that shapes the data exactly as the consumer needs it, pre-joining application records with their relationships and custom attributes in a single response.

The advantage is full access to ServiceNow’s internal scripting capabilities, relationship traversal, and business logic without external [API](#) call overhead. The disadvantage is that development and maintenance happen inside the ServiceNow platform, requiring ServiceNow development expertise and a separate deployment lifecycle. Changes to the [CMDB](#) schema or extraction logic must be coordinated across two platforms. This approach is best suited for organizations with mature ServiceNow development teams that prefer to own the data export logic on the source side.

## Integration Challenges

In implementation phase, the application inventory presents the most significant integration challenge among all data sources because it is the least standardized. Security testing tools converge around common [API](#) patterns and output formats, but each organization structures its application inventory differently: the hierarchy of applications, the granularity of asset mapping, and the attributes captured vary substantially (AXELOS, 2019). This makes the application inventory connector the hardest to generalize and the one most likely to require implementation-specific customization.

Data quality in [CMDBs](#) is a persistent concern (Williams & Gonzalez, 2021). Application records may be incomplete (missing ownership or criticality assignments), outdated (reflecting decommissioned applications still listed as active), or inconsistently maintained across business units. Any integration must account for these quality issues through validation rules and default values that surface problems without blocking the data pipeline.

Despite these difficulties, the application inventory is indispensable. Without the business context it provides, security findings lack the organizational meaning needed for effective prioritization and governance.

### 1.1.4 Cloud Asset Discovery

Major cloud providers offer asset inventory [APIs](#). [Amazon Web Services \(AWS\)](#) Config maintains a continuously updated resource inventory through a [REST API](#). [Azure Resource Graph](#) supports a query language for cross-subscription metadata retrieval. [Google Cloud](#) provides the [Cloud Asset Inventory API](#). All three offer mature [Software Development Kits \(SDKs\)](#) that abstract authentication, pagination, and retry logic. [Kubernetes](#) exposes workload metadata (deployments, pods, container specifications) through its [API](#), enabling the framework to trace from runtime workloads to container images and, through [Continuous Integration and Continuous Delivery \(CI/CD\)](#) metadata, to source repositories.

Cloud asset data serves as the infrastructure counterpart to the business application inventory in the preceding subsections. While the application inventory maps business applications to repositories and teams, cloud asset discovery maps applications to their runtime infrastructure: compute instances, containers, load balancers, and network configurations. Joining these two inventories enables the framework to link security findings from any layer to both code owners and infrastructure owners.

### 1.1.5 Cross-Layer Correlation

The central challenge in asset inventory is correlating identifiers across layers. Security tools that analyze source code identify targets by repository, file path, and code location. Tools that operate on running infrastructure identify targets by host, container, cloud resource, or [URL](#). Bridging these two coordinate systems requires the application-to-asset mapping described in the preceding subsections, which serves as the joining entity linking runtime infrastructure to business applications and their source repositories.

This correlation enables cross-layer analyses: tracing a runtime vulnerability to the code that introduced it, linking a [Web Application Firewall \(WAF\)](#) alert to the responsible application and team, or determining which [Infrastructure as Code \(IaC\)](#) finding corresponds to a live [Cloud Security Posture Management \(CSPM\)](#) detection. Without it, findings from dynamic security tools ([Section 1.4](#)) remain isolated from the development context, limiting their actionability. The data model in [Chapter 2](#) addresses this by defining explicit relationships between application entities, infrastructure assets, and security findings across all layers.

## 1.2 Software Development

Modern software is built collaboratively from source code, organized in repositories, and deployed through automated pipelines (Pressman & Maxim, 2019; Sommerville, 2015). While [Section 1.1](#) established the business context for security findings, the platforms that support software development provide the technical entities around which those findings are organized. Repositories, build pipelines, and issue trackers form the second major data source category for the framework.

These platforms share common integration patterns that simplify connector design. All expose [REST APIs](#) with [JSON](#) responses as the primary integration surface. Authentication relies on token-based mechanisms: OAuth tokens, [Personal Access Tokens \(PATs\)](#), or service account credentials. Pagination follows either offset-based or cursor-based patterns, and all platforms impose rate limits that consumers must handle through backoff strategies. This consistency enables shared connector logic for authentication, pagination, rate limiting, and error recovery across tools.



### 1.2.1 Source Code Management

Chacon and Straub (2014) provide a thorough treatment of Git, covering branching strategies, merging, and distributed workflows, while Skoulikari (2024) offers a more recent visual introduction. Winters et al. (2020) extend this perspective to organizational scale, describing how large engineering teams manage repositories with thousands of contributors. For this thesis, the repository is the primary entity around which security findings are grouped, as detailed in the data model in Chapter 2.

Development teams host their Git repositories on cloud-based [Source Code Management \(SCM\)](#) platforms. GitHub is the dominant platform, with over 150 million developers and adoption by 92% of Fortune 100 companies (GitHub, 2025b). The 2025 Stack Overflow Developer Survey reports GitHub at 81% adoption for code collaboration, followed by GitLab at 36% (Stack Overflow, 2025). Bitbucket and Azure DevOps serve smaller but significant enterprise segments. Most large organizations use more than one platform, whether through acquisitions, team preferences, or regulatory separation, making multi-platform ingestion a practical requirement.

Beyond source code, [SCM](#) platforms expose metadata relevant to security governance. Repository attributes include the primary programming language, creation and last activity dates, visibility settings (public or private), and archive status. Branch protection rules indicate the maturity of the development process. Team and contributor assignments establish ownership relationships that feed into the application-to-team mapping described in [Section 1.1](#).

These platforms share common [API](#) patterns. GitHub exposes two complementary [APIs](#) (GitHub, 2024): a [REST API](#) (v3) with resource-oriented endpoints and page-based pagination, and a [GraphQL API](#) (v4) that enables selective field retrieval with cursor-based pagination. Both use the same authentication mechanisms (OAuth tokens or [PATs](#)) and share a rate limit of 5 000 requests per hour. GitLab offers a similar dual [REST](#)/[GraphQL](#) surface (GitLab, 2024). Azure DevOps provides a [REST API](#) authenticated through Azure Active Directory, with comparable response formats and pagination. Across all platforms, authentication is token-based and responses are [JSON](#)-formatted, enabling shared connector logic for authentication, pagination, and rate limit handling.

### 1.2.2 Continuous Integration and Delivery

[Software Development Lifecycle \(SDLC\)](#) practices have shifted from sequential models to iterative methodologies and DevOps. Forsgren et al. (2018) present empirical evidence that high-performing teams deploy more frequently with shorter lead times, achieved through [CI/CD](#) automation. Kim et al. (2021) complement this with practical guidance for implementing DevOps pipelines at scale. The [CI/CD](#) pipeline, an automated sequence of build, test, and deploy stages, is relevant to this thesis because each stage can integrate

security tools whose findings the framework consolidates.

GitHub Actions is the most widely adopted [CI/CD](#) platform, used by 62% of developers for personal projects and 41% in organizational settings (JetBrains, 2025). Jenkins remains prevalent in enterprises at 28% organizational adoption, followed by GitLab CI at 19%. Notably, 32% of organizations use two or more [CI/CD](#) tools, and 9% use at least three, reinforcing the multi-platform integration challenge observed for [SCM](#) platforms.

[CI/CD](#) platforms provide contextual data about when and how security scans run. Pipeline definitions reveal which security tools are integrated into the build process. Build results indicate whether security gates passed or failed. Execution metadata records which commit was scanned, when the scan ran, and how long it took. This information serves as operational context rather than a primary finding source: knowing that a repository's last security scan ran three months ago, or that a scan consistently fails, signals coverage gaps and tool health that complement the findings from security tools.

Integration patterns vary more across [CI/CD](#) platforms than across [SCM](#) tools. GitHub Actions exposes build data through its [REST API](#) with [JSON](#) responses, consistent with the broader GitHub [API](#) surface. Jenkins uses an [Extensible Markup Language \(XML\)](#)-based [API](#) with a different authentication model. Azure Pipelines follows the Azure DevOps [REST](#) conventions. Despite these differences, the retrieved data is structurally similar: pipeline identifiers, run statuses, timestamps, and associated commit references.

### 1.2.3 Issue Tracking

Issue tracking systems are relevant to the framework because they track remediation status. When a security finding is assigned for remediation, an issue is typically created in the team's tracker, either manually or through automation. The framework must read issue status to determine whether a finding is under active remediation, who is responsible, and whether it has been resolved within the required [Service Level Agreement \(SLA\)](#) window.

Jira is the dominant issue tracking platform in enterprise environments. The 2025 Stack Overflow Developer Survey reports Jira at 46% adoption, behind only GitHub (81%) and ahead of GitLab (36%) for code documentation and collaboration tools (Stack Overflow, 2025). Azure DevOps Work Items serves organizations within the Microsoft ecosystem. GitHub Issues and GitLab Issues provide lightweight built-in tracking tightly coupled with their respective [SCM](#) platforms. As with [SCM](#) and [CI/CD](#) tools, many organizations use multiple trackers across different teams.

These platforms expose [REST APIs](#) with [JSON](#) responses and paginated results. Jira provides [Jira Query Language \(JQL\)](#), a query language for precise filtering by project, status, labels, or custom fields, enabling retrieval of only security-relevant issues without downloading entire project backlogs. GitHub and GitLab expose issues through the same [API](#) surfaces used for repository data. Most trackers also support webhooks for real-time event notifications

when issue status changes, reducing reliance on periodic polling.

The integration with issue trackers is bidirectional. The framework reads remediation status and may also create new issues when findings require attention. This bidirectional flow introduces complexity around idempotency (avoiding duplicate issue creation) and state synchronization (keeping the framework's view consistent with the tracker's current state).

### 1.3 Static Application Security

Application security encompasses the practices and tools for identifying vulnerabilities in software throughout its lifecycle (Anderson, 2020; Stallings & Brown, 2024). McGraw (2006) advocated for security touchpoints spanning the entire development process. The DevSecOps paradigm (Kim et al., 2021; Mack, 2023) realizes this vision by embedding security testing into CI/CD pipelines. *SAST* examines source code, *Software Composition Analysis (SCA)* checks third-party dependencies, secret scanners flag leaked credentials, and *Dynamic Application Security Testing (DAST)* probes running applications. The practical consequence is a high volume of security data from tools with different APIs, output formats, and severity models. The *Open Worldwide Application Security Project (OWASP)* Top 10 classifies common web application risks (OWASP Foundation, 2021), but it does not define a data interchange standard.

Several cross-cutting data standards bring partial consistency. The *Common Vulnerabilities and Exposures (CVE)* system assigns unique identifiers to known vulnerabilities (MITRE Corporation, 2024a). *Common Weakness Enumeration (CWE)* classifies underlying weakness types (MITRE Corporation, 2024b). *Common Vulnerability Scoring System (CVSS)* scores attack vector, complexity, and impact on a 0–10 scale (Forum of Incident Response and Security Teams, 2019). *Exploit Prediction Scoring System (EPSS)* complements *CVSS* with a predictive signal (Jacobs et al., 2021): the probability that a vulnerability will be exploited within 30 days, computed by *Machine Learning (ML)* models. *Cybersecurity and Infrastructure Security Agency (CISA)* maintains the *Known Exploited Vulnerabilities (KEV)* catalog (Cybersecurity and Infrastructure Security Agency, 2024). It adds a third signal: confirmed active exploitation from real-world incident data. Together, *CVSS*, *EPSS*, and *KEV* form a three-signal enrichment model used throughout the framework.

For data interchange, *Static Analysis Results Interchange Format (SARIF)* defines a JSON-based format for static analysis results (OASIS Open, 2024). *CycloneDX* and *Software Package Data Exchange (SPDX)* provide *Software Bill of Materials (SBOM)* schemas (Linux Foundation, 2024; OWASP Foundation, 2024a). *Open Cybersecurity Schema Framework (OCSF)* standardizes security event exchange (OCSF Project, 2024). It targets *Security Information and Event Management (SIEM)* and *Security Orchestration, Automation, and Response (SOAR)* use cases. No single format covers all tool categories. *SARIF* addresses static analysis but not *SCA* or runtime findings. *CycloneDX* and *SPDX* cover software composition, not code-level

vulnerabilities. [OCSF](#) serves security operations, not application security posture. This gap motivates the normalization model in [Chapter 2](#).

The analysis of security tooling is organized into two sections by operational model. This section covers static analysis tools that examine source code and build artifacts without executing the application. Their findings reference repositories, file paths, and line numbers, making them directly attributable to development teams. [Section 1.4](#) covers tools that operate on running applications and deployed infrastructure, where findings reference [URLs](#), hosts, and cloud resources, requiring additional mapping to connect them back to source code. Mobile application security testing is excluded from this analysis, as the framework targets web applications and backend services.

### 1.3.1 Static Application Security Testing

[SAST](#) examines source code, bytecode, or binary code for vulnerabilities without executing the program. Chess and West ([2007](#)) provide the foundations, covering taint analysis, control flow analysis, and pattern matching to detect injection flaws, buffer overflows, and insecure data handling. A key tradeoff is between detection coverage and false positive rates: tools with broader rule sets catch more issues but generate more noise (Chess & West, [2007](#)).

SonarQube provides the most mature integration surface among [SAST](#) tools. Its [REST API](#) offers paginated endpoints with server-side filtering by project, severity, status, and rule, returning [JSON](#) responses (SonarSource, [2024](#)). Beyond individual findings, the [API](#) exposes project-level quality metrics useful for coverage analysis. Semgrep operates as a [Command-Line Interface \(CLI\)](#) tool running user-defined rules against source code, producing [JSON](#) or [SARIF](#) output (Semgrep, [2024](#)). The Semgrep Cloud Platform adds a centralized [API](#) for managing rules and results across projects. Checkmarx exposes a [REST API](#) for scan management and result retrieval, with [SARIF](#) export support.

[SCM](#) platforms also provide built-in [SAST](#) capabilities. GitHub Code Scanning runs CodeQL analysis and exposes results through the code scanning alerts [API](#) (GitHub, [2024](#)). GitLab integrates [SAST](#) as a [CI/CD](#) pipeline job with findings accessible through its security [API](#) (GitLab, [2024](#)). When both platform-native and standalone tools scan the same repository, the resulting overlap requires deduplication in the normalization layer.

Each finding typically includes a rule identifier, the affected file path and line number, a severity rating, the offending code snippet, and remediation guidance. However, severity models differ across tools. SonarQube uses a five-level scale (blocker, critical, major, minor, info). Semgrep and Checkmarx use high, medium, and low, sometimes with a numerical score. This divergence makes severity normalization one of the most important tasks in the framework's transformation layer. The reference implementation integrates SonarQube, Semgrep, and additional [SAST](#) sources as described in [Chapter 3](#).

### 1.3.2 Software Composition Analysis

[SCA](#) identifies known vulnerabilities in third-party dependencies. Modern applications commonly include hundreds of transitive dependencies, creating a large attack surface (Anderson, 2020). Ohm et al. (2020) provide a systematic review of supply chain attacks, including typosquatting, dependency confusion, and malicious package injection. [SCA](#) tools analyze dependency manifests, resolve the full dependency tree, and cross-reference versions against vulnerability databases such as the [National Vulnerability Database \(NVD\)](#).

Snyk provides [REST APIs](#) (legacy v1 and current v3) returning [JSON](#) responses with filtering by severity and project (Snyk, 2024). Dependabot, integrated into GitHub, exposes alerts through the GitHub GraphQL [API](#) (GitHub, 2024). [OWASP Dependency-Track](#) is an open-source platform that ingests [SBOMs](#) in CycloneDX or [SPDX](#) format and identifies vulnerabilities against multiple databases (OWASP Foundation, 2024b). It provides a [REST API](#) for project management and findings retrieval. Many [SCA](#) tools also generate [SBOMs](#) (Linux Foundation, 2024; OWASP Foundation, 2024a), increasingly required by regulatory frameworks (National Telecommunications and Information Administration, 2021). The reference implementation integrates Dependabot, Dependency-Track, and GitLab dependency scanning, as described in [Chapter 3](#).

[SCA](#) output is comparatively well standardized because the underlying data originates from shared public databases. Each finding typically includes the vulnerable dependency name and version, a [CVE](#) identifier, a [CVSS](#) score, the fixed version when available, and exploitability metadata. Despite this, transitive dependency resolution remains a challenge: different tools may resolve the same dependency tree differently, causing one tool to flag a [CVE](#) while another does not. The framework must handle this during deduplication, recognizing that the same [CVE](#) reported by multiple tools against the same repository likely represents a single issue.

[SCA](#) tools also detect license violations (e.g., GPL-licensed code in proprietary applications), producing compliance findings alongside vulnerability data. The [Supply-chain Levels for Software Artifacts \(SLSA\)](#) framework (OpenSSF, 2024) complements [SCA](#) from a different angle: while [SCA](#) checks whether dependencies contain known vulnerabilities, [SLSA](#) verifies the integrity and provenance of build artifacts, ensuring they have not been tampered with during the build process. Both concerns feed into the broader supply chain security picture.

### 1.3.3 Secret Detection

Secret detection tools scan version control history for credentials, [API](#) keys, tokens, and other sensitive material. Once a secret is committed, it persists in Git history even after deletion from the working tree. Tools use two complementary detection approaches. Pattern-based detection matches strings against known credential formats using regular expressions. Entropy-based detection flags strings with unusually high randomness, catching

novel credential formats at the cost of higher false positive rates.

GitLeaks scans repositories against over 150 predefined patterns and produces [JSON](#) output (Scholl, 2024). It runs as a [CLI](#) tool, typically in [CI/CD](#) pipelines or pre-commit hooks. TruffleHog supports over 800 secret types, combines pattern and entropy analysis, and adds live credential verification to confirm whether a detected secret remains valid (Truffle Security, 2024). Both tools operate as [CLI](#) tools without server-side [APIs](#); integration requires executing the tool and parsing its output.

Platform-integrated scanners take a different approach. GitHub Secret Scanning exposes alerts through the GitHub [REST API](#) with webhook notifications for new detections (GitHub, 2024). GitLab Secret Detection runs as a [CI/CD](#) pipeline job and reports results through GitLab’s security dashboard and [API](#) (GitLab, 2024). These tools are simpler to integrate because they use existing platform [API](#) surfaces, but they only cover repositories hosted on their respective platforms.

The primary integration challenge is the absence of a standard output format. Each tool defines its own [JSON](#) schema, and none supports [SARIF](#). Findings include the secret type, file path, commit reference, and sometimes a validity status, but field names and structures vary across tools.

### 1.3.4 Container Image Scanning

Container image scanning examines built container images for vulnerabilities in [Operating System \(OS\)](#) packages, application libraries, and configuration. While [SCA](#) analyzes source-level dependency manifests, container scanners operate on assembled images, detecting [OS](#)-level vulnerabilities and misconfigurations that source-level analysis does not cover.

Trivy is a widely adopted open-source scanner supporting container images, filesystems, and Git repositories (Aqua Security, 2024). It produces [JSON](#) or [SARIF](#) output and integrates into [CI/CD](#) pipelines as a [CLI](#) tool. Commercial alternatives such as Aqua and Prisma Cloud add registry scanning, policy enforcement, and centralized management through [REST APIs](#). Findings typically include the vulnerable package name and version, a [CVE](#) identifier, the fixed version, and the image layer where the package was introduced.

The integration challenge is linking image vulnerabilities to source code repositories. Container images are identified by registry, name, and tag, not by the source code that produced them. Establishing this mapping requires tracing through [CI/CD](#) metadata: which pipeline built the image, from which repository, at which commit.



### 1.3.5 Infrastructure as Code Security

[IaC](#) security tools perform static analysis on infrastructure definitions: Terraform configurations, CloudFormation templates, Kubernetes manifests, and Dockerfiles. They detect misconfigurations before deployment, such as overly permissive access policies, unencrypted storage, exposed ports, and missing logging.

Checkov is a prominent open-source scanner supporting over 1 000 built-in policies across multiple [IaC](#) frameworks (Prisma Cloud by Palo Alto Networks, [2024](#)). [tfsec](#) focuses on Terraform, and [KICS](#) covers a broad range of formats. All operate as [CLI](#) tools producing [JSON](#) or [SARIF](#) output. Findings are file-and-line-based, structurally similar to [SAST](#) output, but target infrastructure configuration rather than application logic.

Kubernetes admission controllers such as OPA/Gatekeeper and Kyverno enforce policies at deploy time, rejecting workloads that violate security constraints. They occupy a middle ground between pre-deployment [IaC](#) scanning and post-deployment [CSPM](#): the policies are defined as code, but enforcement happens at the cluster boundary. Their violation logs provide an additional signal about infrastructure security posture.

These tools complement the runtime monitoring tools discussed in [Section 1.4](#). [CSPM](#) scanners and [Vulnerability Management, Detection, and Response \(VMDR\)](#) platforms detect the same misconfiguration classes after deployment in the live environment. Correlating pre-deployment [IaC](#) findings with post-deployment runtime detections is a challenge the framework addresses through its unified data model.

## 1.4 Dynamic Application Security

While the static analysis tools in [Section 1.3](#) examine source code and build artifacts, the tools in this section operate on running applications and deployed infrastructure. The cross-cutting data standards introduced in [Section 1.3](#), particularly [CVE](#), [CVSS](#), [EPSS](#), and [KEV](#), apply equally to dynamic findings. The cross-layer correlation challenge that connects these findings to business applications is addressed in [Section 1.1.5](#).

### 1.4.1 Dynamic Application Security Testing

[DAST](#) tests running applications by sending crafted [Hypertext Transfer Protocol \(HTTP\)](#) requests and analyzing responses for vulnerability indicators (Stuttard & Pinto, [2011](#)). Unlike [SAST](#), which works on source code, [DAST](#) requires a deployed application and detects vulnerabilities that manifest only at runtime, such as authentication bypass, server misconfiguration, and certain injection flaws.

OWASP Zed Attack Proxy (ZAP) is the most widely used open-source [DAST](#) tool, providing a [REST API](#) for scan management and result retrieval with [JSON](#) and [XML](#) output (OWASP Foundation, 2024c). Burp Suite exposes a [REST API](#) in its Enterprise edition; the Professional edition is a desktop tool without programmatic access. Findings from both tools include the target [URL](#), the affected [HTTP](#) parameter, the vulnerability type mapped to [CWE](#) identifiers (MITRE Corporation, 2024b), exploitation evidence, and a confidence rating.

The core integration challenge is that [DAST](#) findings are [URL](#)-based, not code-based. A [SAST](#) finding points to a file and line in a repository; a [DAST](#) finding points to a [URL](#) endpoint. Mapping [URL](#)-based findings to source repositories requires deployment metadata or application inventory data from [Section 1.1](#). Without this mapping, [DAST](#) findings can be attributed to applications but not to development teams or code locations.

### 1.4.2 Penetration Testing

Manual penetration testing traditionally produces no machine-readable output. External firms deliver results as narrative [Portable Document Format \(PDF\)](#) or Word reports, requiring a structured upload mechanism (e.g., a [JSON](#) or [Comma-Separated Values \(CSV\)](#) template) for security teams to transcribe key fields. [ML](#)-based document parsing could automate extraction, though the inconsistent formatting of penetration test reports makes this non-trivial.

Emerging platforms such as XBOW use autonomous [Artificial Intelligence \(AI\)](#) agents to perform penetration testing at scale, delivering validated findings with reproducible exploit evidence (XBOW, 2024). These platforms offer [API](#) access for test orchestration and integrate with security data ecosystems, moving penetration testing toward the structured, automatable data exchange that other tool categories already provide.

Despite low frequency, penetration testing findings are often among the most critical, representing validated exploitable vulnerabilities. The framework must accommodate both manual uploads and automated [API](#) ingestion to cover the full spectrum of penetration testing practices.

### 1.4.3 Web Application Firewalls

[WAFs](#) operate at the network perimeter, inspecting inbound [HTTP](#) traffic against rule sets such as the [OWASP Core Rule Set](#). They log blocked and flagged requests, recording the matched rule, request details, and action taken. Most commercial [WAFs](#) ([AWS WAF](#), [Cloudflare](#), [Akamai](#)) expose [REST APIs](#) for log retrieval and configuration management.

These platforms typically bundle [Distributed Denial of Service \(DDoS\)](#) protection alongside [WAF](#) capabilities. [DDoS](#) mitigation services filter volumetric and application-layer attacks,



logging traffic patterns, attack signatures, and mitigation actions. This telemetry provides availability impact data that complements the vulnerability-focused findings from other tool categories.

WAFs and DDoS protection generate high-volume event data rather than discrete findings. The framework ingests aggregated attack patterns and anomaly indicators rather than individual request logs, focusing on the subset relevant to application security posture assessment.

#### 1.4.4 Runtime Application Self-Protection

Runtime Application Self-Protection (RASP) tools instrument the application runtime, detecting attacks such as Structured Query Language (SQL) injection and path traversal from within the application context. Unlike WAFs, which operate at the network perimeter, RASP can correlate attacks with specific application functions and code paths. This produces richer contextual data per event, but RASP adoption remains limited compared to WAFs, and integration surfaces vary across vendors.

Like WAFs, RASP tools produce event streams rather than discrete findings. The integration approach is similar: the framework consumes aggregated indicators rather than raw event logs.

#### 1.4.5 Runtime API Security

Traditional DAST tools actively probe applications with crafted requests. Runtime API security platforms take a different approach: they passively monitor live API traffic, build behavioral baselines, and detect anomalies that indicate abuse (OWASP Foundation, 2023). This continuous monitoring catches threats that active scanning misses, including business logic abuse, account takeover patterns, and data exfiltration through legitimate API endpoints.

Salt Security pioneered this category, applying AI-based behavioral analysis to API traffic to discover shadow APIs, detect anomalies, and identify attack patterns (Salt Security, 2024). Noname Security, acquired by Akamai in 2024 (Akamai Technologies, 2024), offers similar capabilities: API discovery, vulnerability detection, and runtime protection integrated into Akamai's edge platform. Both expose REST APIs for configuration and findings retrieval.

Runtime API security produces event-oriented data similar to WAFs and RASP, but with richer API-specific context: the affected endpoint, request/response patterns, the deviation from baseline behavior, and the attack classification. The framework treats these as a continuous finding source, ingesting aggregated indicators rather than raw traffic data.

### 1.4.6 Vulnerability Management and Cloud Posture

**VMDR** tools scan infrastructure for **OS** vulnerabilities, missing patches, and exposed services. Qualys exposes data through a **REST API** with **XML** responses. Wiz uses a GraphQL **API** for selective queries across cloud environments. These tools produce findings tied to infrastructure identifiers (hostnames, IP addresses, cloud resource **Amazon Resource Names (ARNs)**), not to source code locations.

**CSPM** tools complement **VMDR** by evaluating cloud configurations against compliance benchmarks such as **Center for Internet Security (CIS)** Benchmarks and organizational policies. Findings indicate misconfigurations (public S3 buckets, overly permissive **Identity and Access Management (IAM)** roles, unencrypted storage) rather than software vulnerabilities. **CSPM** overlaps with **IaC** security ([Section 1.3.5](#)): both detect the same misconfiguration classes, but **IaC** tools scan before deployment while **CSPM** scans the live environment. Correlating pre-deployment and post-deployment findings is a key integration challenge.

### 1.4.7 Source Integration Summary

[Table 1.1](#) summarizes the integration characteristics across all source categories analyzed in this chapter.

**Table 1.1**

*Data Source Integration Characteristics*

| Source Category       | API             | Format     | Frequency  | Standardization |
|-----------------------|-----------------|------------|------------|-----------------|
| Application Inventory | REST            | JSON       | Low        | Low             |
| Cloud Assets          | REST            | JSON       | Continuous | Medium          |
| SCM Platforms         | REST/GraphQL    | JSON       | Medium     | High            |
| Issue Trackers        | REST            | JSON       | Medium     | Medium          |
| CI/CD Platforms       | REST/XML        | JSON/XML   | Medium     | Medium          |
| SAST                  | REST/CLI        | JSON/SARIF | On-demand  | Medium          |
| SCA                   | REST/GraphQL    | JSON       | Continuous | High            |
| Secret Scanning       | CLI/REST        | JSON       | On-demand  | Low             |
| Container Scanning    | CLI             | JSON/SARIF | On-demand  | Medium          |
| IaC Scanning          | CLI             | JSON/SARIF | On-demand  | Medium          |
| DAST                  | REST/CLI        | JSON/XML   | On-demand  | Medium          |
| Penetration Testing   | None            | PDF        | Periodic   | None            |
| WAF/DDoS              | REST            | JSON       | Continuous | Low             |
| RASP                  | Vendor-specific | JSON       | Continuous | Low             |
| API Security          | REST            | JSON       | Continuous | Low             |
| VMDR/CSPM             | REST/GraphQL    | JSON/XML   | Continuous | Medium          |

Several patterns emerge from this comparison. [REST APIs](#) with [JSON](#) responses are the dominant integration surface, but [XML](#) responses, GraphQL queries, [CLI](#) output parsing, and manual uploads are all necessary. Standardization varies: [SCA](#) benefits from the [CVE/NVD](#) ecosystem, [SAST](#) is partially standardized through [SARIF](#), and secret scanning has no standard format. Update frequencies range from continuous dependency monitoring to periodic penetration tests. These characteristics define what the ingestion layer described in [Chapter 2](#) must accommodate.

## 1.5 Data Engineering

The preceding sections analyzed the sources of application security data: asset inventories, development platforms, and security testing tools across static and dynamic categories. Consolidating these heterogeneous sources into a unified analytical platform is fundamentally a data integration problem. This section surveys the architectural paradigms and engineering patterns relevant to solving it, independent of any specific platform.

### 1.5.1 Data Platform Architecture

The literature documents a progression from data warehouses through data lakes to the lakehouse paradigm. DAMA International ([2017](#)) provide a comprehensive body of knowledge for data management, establishing the vocabulary and principles referenced throughout this section. Codd ([1970](#)) established the relational model as the theoretical foundation. The data warehouse, formalized by Inmon ([2005](#)), stores subject-oriented, integrated data for analytical decision-making using a top-down normalized approach. Kimball and Ross ([2013](#)) take a complementary bottom-up path with dimensional modeling: star schemas that organize data into fact and dimension tables optimized for queries. Both approaches assume structured sources with stable schemas, a condition that does not hold for heterogeneous security tool output.

The data lake (Miloslavskaya & Tolstoy, [2016](#)) stores raw data in its native format, deferring schema definition to consumption time (schema-on-read). This accommodates format diversity but introduces governance challenges. Without proper management, data lakes risk becoming “data swamps” where data is abundant but unusable (DAMA International, [2017](#)).

The lakehouse architecture (Armbrust et al., [2021](#)) combines data lake flexibility with warehouse governance by introducing open table formats that add [Atomicity](#), [Consistency](#), [Isolation](#), [Durability \(ACID\)](#) transactions, schema enforcement, and time travel on top of lake storage (Lee et al., [2024](#); Thalpati, [2024](#)). The lakehouse paradigm suits the application security problem: it accommodates diverse, semi-structured tool output while provid-

ing the governance and query performance required for both analytical dashboards and operational lookups.

The medallion architecture (Strengtholt, 2025) organizes lakehouse data into three layers:

- **Bronze:** Raw data with minimal transformation, preserving original source schemas.
- **Silver:** Cleaned, validated, and normalized data conforming to a unified schema.
- **Gold:** Consumption-ready aggregated metrics and enriched datasets.

This layered approach maps naturally to the security data integration problem. The bronze layer captures raw tool output in source-native schemas. The silver layer normalizes findings across tools into a vendor-agnostic model. The gold layer computes the aggregated risk metrics and enriched datasets that stakeholders consume.

A security data platform must serve two distinct access patterns. [Online Analytical Processing \(OLAP\)](#) queries power dashboards and trend analysis, operating over large aggregated datasets where query throughput matters more than write latency. [Online Transaction Processing \(OLTP\)](#) access supports operational workflows such as issue tracker integration and real-time risk lookups, where low-latency reads and writes to individual records are essential. The framework must accommodate both, a requirement that influences the serving architecture designed in [Chapter 2](#).

### 1.5.2 Data Integration Patterns

Reis and Housley (2022) cover data engineering fundamentals, including the distinction between [Extract, Transform, Load \(ETL\)](#) and [Extract, Load, Transform \(ELT\)](#) paradigms. [ETL](#) transforms data before loading, requiring upfront schema knowledge and a dedicated transformation layer between source and target. [ELT](#) loads raw data first and transforms it in place, aligning well with lakehouse architectures where distributed compute engines perform transformations at scale (Thalpati, 2024). For security data integration, [ELT](#) is the natural fit: tool output varies widely in structure and evolves frequently, making upfront transformation brittle.

Full re-ingestion does not scale for enterprise environments with thousands of repositories and dozens of security tools. Incremental ingestion pulls only records that are new or changed since the last synchronization. The high-water mark pattern tracks a timestamp or cursor per source, resuming extraction from the last known position. [Change Data Capture \(CDC\)](#) captures changes at the source level, propagating inserts, updates, and deletes as events (Reis & Housley, 2022). Both approaches reduce [API](#) call volume and processing time, enabling more frequent pipeline runs without proportional resource growth.

Source tools change their [APIs](#) and output formats regularly. A rigid schema definition would break pipelines with every upstream change. Schema-on-read stores raw data in its original structure and applies schema interpretation at query time. Additive schema

evolution allows new fields to be absorbed without breaking existing queries or downstream consumers (Lee et al., 2024). For security tool integration, this flexibility is essential: tools add new finding attributes, rename fields, and change enumerations across versions.

Pipeline re-runs must produce the same result regardless of how many times they execute. This idempotency property is essential for handling retries after partial failures and for reprocessing historical data without creating duplicates. Data quality enforcement complements idempotency through validation at each pipeline layer: schema conformance at ingestion, value range checks during normalization, and referential integrity at aggregation. Records that fail validation are routed to quarantine tables rather than silently dropped or propagated downstream (DAMA International, 2017). This quarantine pattern ensures zero silent data loss: every ingested record is either successfully processed or isolated with a documented failure reason.

### 1.5.3 Domain Data Model

The security domains analyzed in [Section 1.1](#) through [Section 1.4.7](#) produce data about a common set of entities and relationships. Before designing physical schemas, a vendor-agnostic conceptual model must define what the framework represents. This subsection identifies the core entities, their relationships, and the consumption patterns that drive the serving layer.

Five primary entity types form the backbone of the domain model:

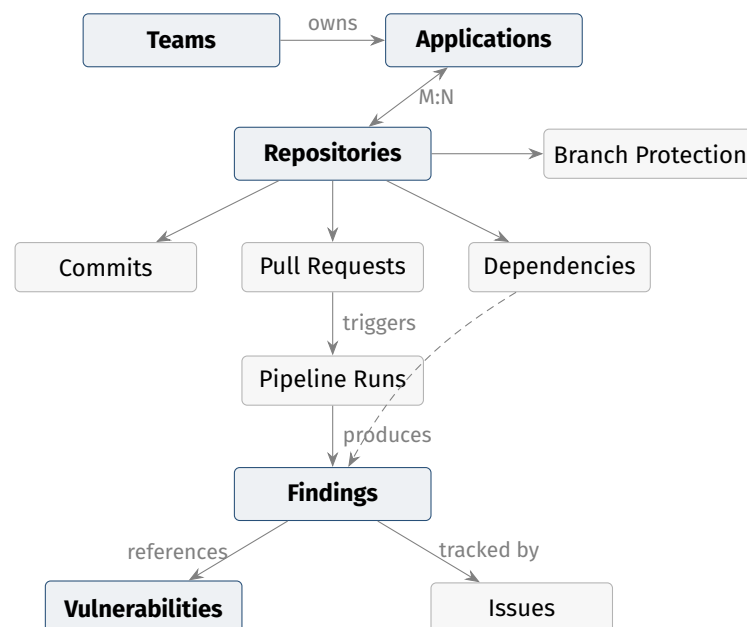
- **Applications** carry business context from asset inventories ([Section 1.1](#)): name, owning team, criticality tier, lifecycle status, and compliance scope. An application represents a business-level unit that may span multiple repositories and deployment environments.
- **Repositories** represent technical assets from [SCM](#) platforms ([Section 1.2](#)): code location, primary language, visibility, activity indicators, and archive status. The repository is the primary unit around which security findings are grouped.
- **Findings** capture security issues from any tool category analyzed in [Section 1.3](#) and [Section 1.4](#): severity, status, source tool, detection timestamp, affected code location, and rule or check identifier. Each finding traces to a specific tool scan.
- **Vulnerabilities** are [CVE](#)-identified issues enriched with three signals: [CVSS](#) severity from the [NVD](#), exploitation probability from [EPSS](#), and confirmed exploitation status from [CISA KEV](#). Multiple findings from different tools may reference the same vulnerability.
- **Teams** provide organizational ownership, linking personnel to applications and remediation responsibilities. Team structure enables filtering and aggregation by organizational unit.

Several additional entity types provide the development process and supply chain context

necessary for actionable security analytics:

- **Commits** record individual code changes with author, timestamp, and affected files. They link findings to specific code versions and establish authorship for attribution.
- **Pull requests** represent code change proposals where security scans typically execute. Scan results are often reported per pull request, making it a natural grouping unit for new findings.
- **Pipeline runs** capture **CI/CD** execution records: which security tools ran, when, against which commit, and whether security gates passed. They indicate scan coverage and tool health across repositories.
- **Issues** track remediation work items in systems such as Jira. Linking findings to issues enables **Mean Time to Remediate (MTTR)** calculation and **SLA** compliance monitoring.
- **Dependencies** represent third-party libraries identified through **SCA**: package name, version, license, and associated **CVE** identifiers. They connect **SCA** findings to the specific library version that introduced the vulnerability.
- **Branch protection policies** capture governance configurations from **SCM** platforms: required reviewers, mandatory status checks, and merge restrictions. They serve as indicators of development process maturity and security hygiene at the repository level.

Figure 1.1 illustrates the entity relationships. Primary entities (bold borders) form the core analytical model; supporting entities provide development process and supply chain context.



**Figure 1.1**  
*Conceptual Domain Model*

The application-to-repository mapping is the most critical relationship: it links technical findings to business context, enabling risk-aware prioritization. This relationship is many-

to-many; shared libraries serve multiple applications, and microservice applications span multiple repositories. Teams own applications, establishing the chain from organizational accountability through business applications to technical assets and their findings.

Development process entities add temporal and workflow context to this core chain. Repositories contain commits and pull requests, which trigger pipeline runs and produce findings. Findings that reference [CVE](#) identifiers link to vulnerability records, enabling enrichment with external intelligence. Dependencies bridge repositories and vulnerabilities: an [SCA](#) finding links a specific library version in a repository to a known [CVE](#). Issues track the remediation of one or more findings, closing the loop from detection to resolution.

The entity structure maps naturally to dimensional modeling patterns (Kimball & Ross, 2013). Findings function as fact records: measurable events with severity, detection date, and tool source. Applications, repositories, and teams serve as dimension records that provide descriptive context for analysis. Supporting entities such as commits, pull requests, and pipeline runs add temporal and process dimensions. This alignment means standard analytical techniques (filtering by dimension, aggregating facts, computing trends over time) apply directly to the security domain. When multiple tools detect the same underlying issue, cross-tool deduplication collapses related findings while preserving traceability to each source report.

Different stakeholders consume this data through different access patterns. Application owners need aggregated risk dashboards showing finding counts by severity, remediation progress, and compliance status for their applications. Developers need actionable findings with code-level context, delivered through issue trackers. Security experts need drill-down access to raw findings, audit trails, and cross-tool correlation for triage. Leadership needs executive metrics: [MTTR](#), [SLA](#) compliance rates, vulnerability density trends, and portfolio-level comparisons. These consumption patterns span the [OLAP](#) and [OLTP](#) serving requirements identified in [Section 1.5.1](#) and drive the serving architecture designed in [Chapter 2](#).

## 1.6 Related Work and Gap Analysis

Despite the maturity of individual security tools analyzed in the preceding sections, no standard approach exists for consolidating their output into a unified data platform. Organizations face a fragmented landscape of partial solutions, each addressing a subset of the integration challenge. This section surveys existing approaches, evaluates the closest platform-native offering, and identifies the research gap this thesis addresses.



### 1.6.1 Existing Approaches

Gardner et al. (2023) define [Application Security Posture Management \(ASPM\)](#) as a category of tools that continuously manage application risk through the collection, analysis, and prioritization of security issues across the software lifecycle. Commercial platforms such as Apiiro, Cyscale, ArmorCode, and Snyk AppRisk aggregate findings from multiple tools into unified dashboards with correlation and risk scoring. However, they are proprietary, operate as black boxes, and create vendor lock-in: organizations cannot customize data pipelines, apply their own [ML](#) models, or extend integrations beyond vendor-supported options.

Platform-native security features offer another partial solution. GitHub Advanced Security provides code scanning via CodeQL, secret scanning, and dependency review within the GitHub ecosystem (GitHub, 2025a). GitLab bundles [SAST](#), [DAST](#), dependency scanning, container scanning, and a security dashboard into its platform (GitLab, 2025). Both approaches are ecosystem-locked: they aggregate findings only from their own scanners. Organizations using multiple [SCM](#) platforms or third-party security tools cannot consolidate findings through these native dashboards.

DefectDojo (DefectDojo, 2024) is the most prominent open-source alternative, supporting imports from over 150 tools through format-specific parsers. However, it was designed as a vulnerability management interface, not a data platform. Its PostgreSQL backend limits enterprise-scale analytics, and advanced [API](#) connectors are available only in the commercial offering. [OCSF](#) (OCSF Project, 2024) standardizes security event schemas for [SIEM](#) and [SOAR](#) use cases but does not model application-level entities such as repositories, applications, or development teams.

In practice, many enterprises build ad-hoc integrations: custom scripts that pull data from security tool [APIs](#), [ETL](#) pipelines that load findings into data warehouses, and dashboards assembled from manual exports. These solutions are fragile. They lack schema governance, break when source [APIs](#) change, and encode institutional knowledge in unmaintained code. Each organization reinvents the same integration patterns without reusable abstractions or standard architectures.

### 1.6.2 Databricks Lakewatch

Databricks announced Lakewatch in March 2026 as an open, agentic [SIEM](#) built natively on the Databricks lakehouse (Databricks, 2026a). The platform unifies security, [Information Technology \(IT\)](#), and business data in a single governed environment powered by Delta Lake and Unity Catalog. Key capabilities include agentic triage ([AI](#) agents that parse and enrich telemetry across formats), Detection-as-Code (version-controlled detection rules), and an open ecosystem with integrations from security vendors including Wiz, Palo Alto Networks, Okta, and Zscaler.



Lakewatch and the framework proposed in this thesis address different security domains. Lakewatch focuses on runtime security operations: threat detection, incident response, alert triage, and log analytics. The framework focuses on application security posture: vulnerability findings from [SAST](#), [SCA](#), [DAST](#), secret scanning, container scanning, and [IaC](#) tools, combined with remediation tracking and risk aggregation. Their data sources differ accordingly: Lakewatch ingests security logs and telemetry, while the framework ingests security tool findings and application metadata.

The two systems are complementary rather than competing. Shared lakehouse infrastructure enables data exchange: Lakewatch could consume the framework’s application risk scores as enrichment context for incident triage, while the framework could ingest Lakewatch’s runtime detections as an additional finding source. Together, they cover both proactive posture management and reactive threat detection.

### 1.6.3 Comparative Analysis and Research Gap

[Table 1.2](#) compares the surveyed approaches against criteria derived from the domain analysis in [Section 1.1](#) through [Section 1.5](#). The criteria reflect requirements that emerged from analyzing security data sources, integration patterns, and stakeholder consumption needs.

**Table 1.2**

*Comparison of Existing Approaches*

| Criterion                | ASPM | Platform-native | DefectDojo | Lakewatch | This thesis |
|--------------------------|------|-----------------|------------|-----------|-------------|
| Open-source              | –    | –               | Yes        | –         | Yes         |
| Vendor-agnostic model    | –    | –               | Partial    | –         | Yes         |
| Extensible connectors    | –    | –               | Yes        | Partial   | Yes         |
| Lakehouse architecture   | –    | –               | –          | Yes       | Yes         |
| OLAP + OLTP serving      | –    | –               | –          | –         | Yes         |
| Data quality enforcement | –    | –               | –          | –         | Yes         |
| AppSec-specific model    | Yes  | Partial         | Yes        | –         | Yes         |
| Enterprise scalability   | Yes  | Partial         | –          | Yes       | Yes         |

No existing approach satisfies all criteria. Commercial [ASPM](#) platforms are proprietary and inflexible, preventing organizations from customizing pipelines or applying their own ana-

lytical models. Platform-native aggregations are ecosystem-locked and cannot consolidate findings across tool boundaries. DefectDojo lacks the data architecture for enterprise-scale analytics. Lakewatch addresses a different security domain: runtime operations rather than application security posture. Academic literature on application security consolidation as a data engineering problem is limited, with most work focusing on detection techniques rather than cross-tool integration at enterprise scale.

This thesis fills the gap with an open, vendor-agnostic framework that treats application security consolidation as a data engineering problem. The framework applies lakehouse architecture and medallion data organization to ingest heterogeneous security tool output, normalize it into a unified domain model, and serve it through both [OLAP](#) and [OLTP](#) interfaces. The design is extensible: adding a new data source requires only a connector module, without changes to the core pipeline. [Chapter 2](#) presents the framework design, and [Chapter 3](#) demonstrates its implementation.

## 2. Framework

[Chapter 1](#) analyzed the sources, patterns, and gaps in enterprise application security data integration. This chapter presents the proposed framework: a reusable blueprint that defines the architecture, data model patterns, connector abstractions, and extension points an organization follows to build its own implementation. The framework is prescriptive but platform-aware; it targets the Databricks lakehouse but separates general patterns from platform-specific choices. [Chapter 3](#) demonstrates a concrete instance of this blueprint.

### 2.1 Solution Architecture

This section presents the framework’s architecture at three levels: technology stack, component structure, and data layer organization. The architecture addresses the requirements identified across the domain analysis: heterogeneous source ingestion, vendor-agnostic normalization, dual [OLAP/OLTP](#) serving, and extensibility for new data sources and analytics.

#### 2.1.1 Technology Stack

The framework targets the Databricks platform on [AWS](#). A key architectural property of the platform is the separation of storage and compute: data resides in cloud object storage while processing engines scale independently, enabling cost-efficient handling of the bursty, heterogeneous workloads typical of security data integration (Databricks, [2026a](#)). This subsection maps platform components to framework roles.

#### Delta Lake

Delta Lake is an open table format that adds [ACID](#) transactions, schema enforcement, and time travel to cloud object storage (Lee et al., [2024](#)). It provides the storage layer for all three medallion tiers. Schema evolution support allows bronze tables to absorb new source fields without pipeline changes. Time travel enables auditing and rollback, critical for a security data platform where data integrity and reproducibility are non-negotiable. Armbrust et al. ([2021](#)) demonstrated that the lakehouse architecture built on Delta Lake achieves both warehouse-grade governance and data lake flexibility, the combination required by the heterogeneous source landscape analyzed in [Section 1.4.7](#).

## Unity Catalog

Unity Catalog provides unified data governance across the lakehouse (Databricks, 2025b). It manages a three-level namespace (`catalog.schema.table`) that maps directly to the medallion layer organization: one catalog per environment, schemas for each source (bronze) and each domain (silver, gold). Fine-grained access control enforces least-privilege access to security data, which often contains sensitive vulnerability details. Automated lineage tracking records data flow from source through transformations to consumption, supporting auditability requirements. Column-level tags enable classification of sensitive fields such as [CVE](#) descriptions and remediation guidance.

## Lakeflow Declarative Pipelines

[Lakeflow Declarative Pipelines \(LDP\)](#) is the pipeline orchestration framework for defining batch and streaming transformations (Databricks, 2025a). Pipelines are declared as Python or [SQL](#) functions with explicit dependencies; the framework resolves execution order automatically. Data quality expectations are embedded directly in pipeline definitions as declarative constraints (e.g., “severity must be one of critical, high, medium, low”). Records that violate expectations are quarantined without stopping the pipeline, implementing the quarantine pattern described in [Section 1.5.2](#). LDP handles incremental processing natively through change data feed, reducing reprocessing cost for large datasets.

## Lakebase

Lakebase is a serverless PostgreSQL database that provides the [OLTP](#) serving layer (Databricks, 2026b). It shares the underlying storage layer with the lakehouse, enabling gold-layer tables to be exposed as [OLTP](#)-queryable tables without data duplication. This architecture satisfies the dual serving requirement identified in [Section 1.5.1](#): [OLAP](#) queries run against [SQL](#) warehouses for dashboards and trend analysis, while [OLTP](#) queries run against Lakebase for low-latency operational workloads such as issue tracker integration, remediation state lookups, and serving layer [APIs](#). Lakebase supports scale-to-zero, reducing cost for bursty workloads. Instant database branching creates isolated copies for development and testing without duplicating data.

## Platform Synergies

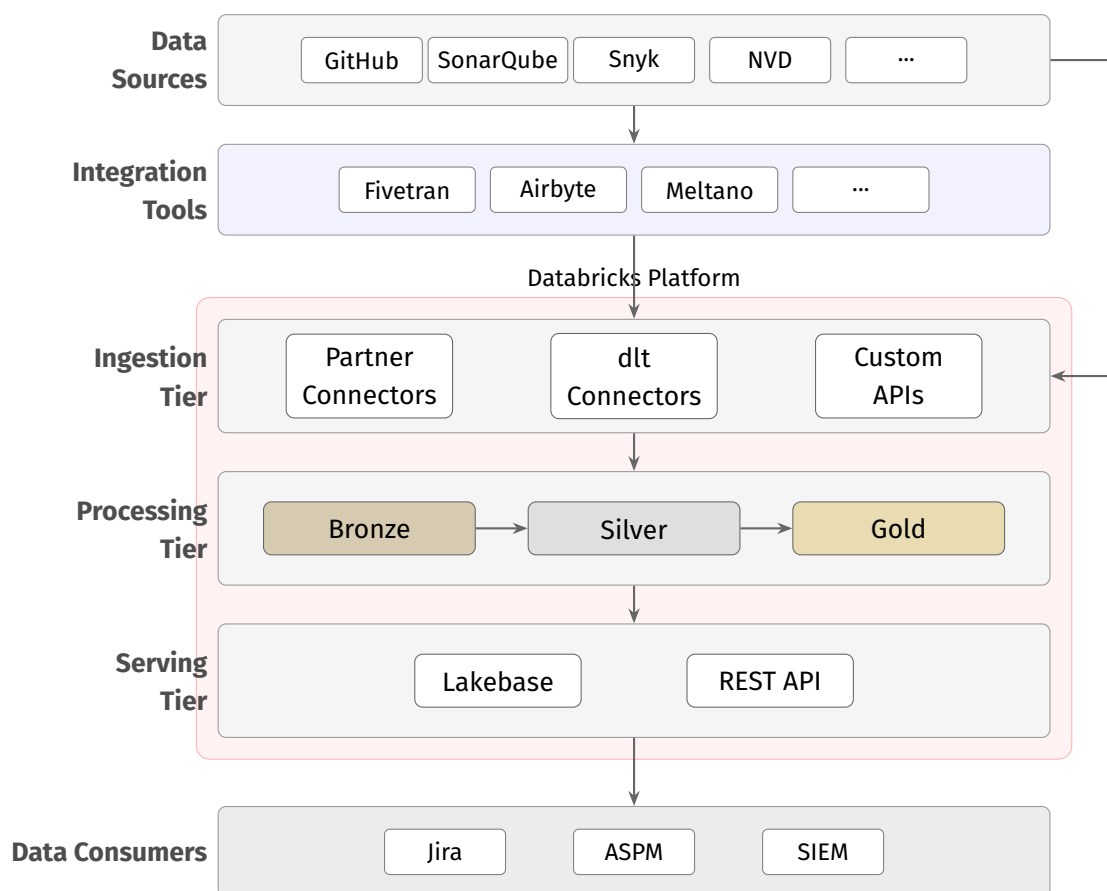
Running all components on a single platform provides governance, lineage, and compute benefits that would require significant integration effort across disparate tools. Unity Catalog governs access uniformly from bronze ingestion through gold serving. Lineage tracks data from source [API](#) through transformations to dashboard queries. The shared

compute and storage model eliminates data movement between analytical and operational stores.

The platform also enables integration with Lakewatch, the Databricks [SIEM](#) analyzed in [Section 1.6.2](#). Both systems share Delta Lake storage and Unity Catalog governance. The framework's gold-layer outputs (application risk scores, remediation status) can serve as enrichment signals for Lakewatch threat detection, while Lakewatch's runtime security events could feed back into the framework as an additional finding source. This bidirectional potential reinforces the complementarity identified in the related work analysis.

## 2.1.2 Component Design

With the platform components established, the framework organizes into five tiers, illustrated in [Figure 2.1](#). Data flows top-to-bottom from sources through the platform to consumers.



**Figure 2.1**  
*Component Design*

**Data sources** are the external systems analyzed in [Chapter 1](#): application inventories, [SCM](#) platforms, [CI/CD](#) tools, security scanners, and vulnerability databases. They are outside the

framework's boundary; the framework consumes their [APIs](#) but does not control them.

**The ingestion tier** hosts connectors that extract data from sources and land it in the bronze layer. Three connector categories serve different integration scenarios. Partner connectors use pre-built integrations from the platform ecosystem. [Data load tool \(dlt\)](#) connectors use the open-source [dlt](#) library for custom [REST API](#) extraction. Custom connectors handle non-standard sources such as [CLI](#)-based scanners or file uploads. All connectors share common concerns: authentication, pagination, rate limiting, and incremental state management. The connector framework in [Section 2.3](#) defines these patterns.

**The processing tier** implements the medallion architecture ([Section 2.1.3](#)). Delta Lake provides the storage layer; [LDP](#) orchestrates the transformations. Bronze stores raw ingested data in source-native schemas. Silver normalizes it into the vendor-agnostic entity and finding model defined in [Section 1.5.3](#). Gold computes aggregated metrics, [ML](#)-enriched scores, and consumption-ready datasets. Unity Catalog governs access and tracks lineage across all three layers.

**The serving tier** exposes processed data to downstream consumers. [SQL](#) warehouses serve [OLAP](#) queries for dashboards and analytics. Lakebase serves [OLTP](#) workloads: low-latency lookups, issue tracker integration, and operational [APIs](#). A [REST API](#) layer provides [HTTP](#) access to the [OLTP](#) store.

**Data consumers** are external systems that read from the serving tier: issue trackers (Jira, ServiceNow), [ASPM](#) dashboards, [SIEM/SOAR](#) platforms, and custom reporting tools. Like data sources, consumers are outside the framework boundary.

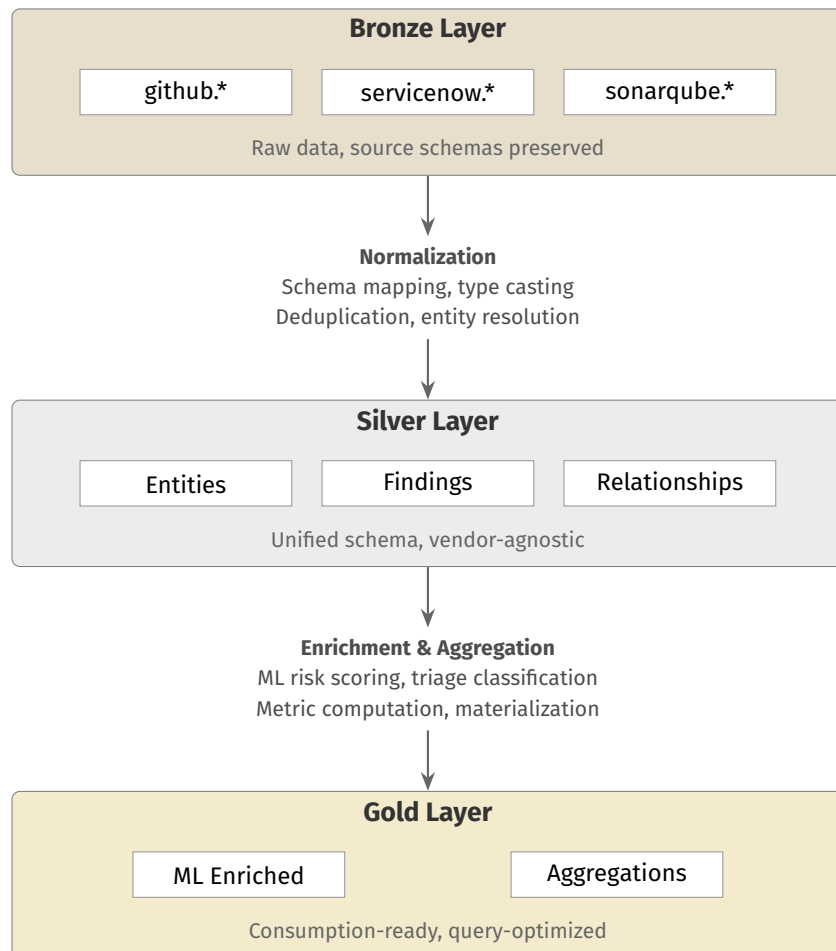
A direct bypass path allows data sources to reach the ingestion tier without an intermediate integration tool, for cases where connectors call source [APIs](#) directly. This path is common for security scanners whose output is consumed through native [REST APIs](#) or [CLI](#) tooling.

### 2.1.3 Medallion Architecture

The processing tier applies the medallion architecture pattern introduced in [Section 1.5.1](#). Each layer is implemented as a Delta Lake schema within a single Unity Catalog, providing unified governance and cross-layer lineage tracking. [Figure 2.2](#) illustrates the layer structure and the transformations between them.

#### Bronze Layer

The bronze layer stores raw data from each source system with minimal transformation. Each connector writes to its own schema (e.g., `github`, `servicenow`, `sonarqube`), preserving the source's native data structure. Records are appended with standard metadata columns:



**Figure 2.2**  
*Medallion Layer Design*

ingestion timestamp, source system identifier, and batch identifier. No business logic is applied at this layer; the goal is a faithful, auditable copy of source data.

Bronze tables use a schema-on-read approach. New fields from source [API](#) changes are absorbed through additive schema evolution without breaking existing pipelines. Partitioning by ingestion date enables efficient time-range queries and supports retention policies. Records that fail structural validation at ingestion (malformed [JSON](#), missing required fields) are routed to per-source quarantine tables with diagnostic metadata.

## Silver Layer

The silver layer is the system of record. Bronze-to-silver transformations normalize heterogeneous source data into the vendor-agnostic domain model defined in [Section 1.5.3](#). Three table categories occupy this layer:

- **Entity tables** store normalized dimension data: applications, repositories, teams, commits, pull requests, pipeline runs, dependencies, and branch protection policies. Each entity table uses a surrogate key, a natural key from the source system, and `valid_from/valid_to` timestamps for change tracking.
- **Finding tables** store normalized security findings as fact records. Each finding carries a severity mapped to the canonical scale, a lifecycle status, the source tool identifier, and references to the affected entity (repository, application). Finding tables are organized by category: [SAST](#), [SCA](#), secrets, [DAST](#), containers, and [IaC](#).
- **Relationship tables** store many-to-many mappings: application-to-repository, finding-to-[CVE](#), and cross-tool deduplication links.

The silver layer applies the data quality patterns from [Section 1.5.2](#): severity harmonization, entity normalization, timestamp standardization, and deduplication. [LDP](#) expectations enforce constraints declaratively; records that violate expectations are quarantined rather than propagated.

## Gold Layer

The gold layer computes consumption-ready datasets from silver data. Two categories of gold tables serve distinct purposes:

- **Aggregation tables** compute metrics at defined grains: application risk scores, team remediation rates, [MTTR](#) by severity, [SLA](#) compliance percentages, and time-series trend roll-ups. These tables power the dashboards and executive reports consumed through the [OLAP](#) serving path.
- **ML-enriched tables** store model outputs: composite risk scores, false positive predictions, and remediation time estimates. These scores augment the aggregation tables



with predictive signals. The [ML](#) workflow patterns are detailed in [Section 2.4.2](#).

Gold tables use incremental refresh where possible: new silver records trigger recomputation of only the affected aggregation partitions. Full refresh is reserved for metrics that require global recomputation, such as cross-application percentile rankings.

## 2.2 Data Model

This section maps the conceptual domain model from [Section 1.5.3](#) to physical table designs across the three medallion layers. It defines reusable schema patterns first, then applies them to produce the concrete entity, finding, and aggregation tables the framework provides.

### 2.2.1 Schema Patterns

Schema patterns are reusable templates that standardize how tables are created at each medallion layer. They ensure consistency across the data model and lower the barrier for extending it with new entities or sources. Four patterns cover the framework's needs.

#### Bronze Pattern

Every bronze table follows the same structural convention. The source's native payload is stored alongside a fixed set of metadata columns:

- `_ingestion_timestamp` — when the record was ingested ([Coordinated Universal Time \(UTC\)](#)).
- `_source_system` — identifier of the originating tool or platform.
- `_batch_id` — unique identifier for the ingestion run, supporting idempotent replays.
- `_raw_payload` — the original [JSON](#) response or record, preserved for auditability.

Additional columns from the source's native schema are included alongside the raw payload through schema-on-read. New fields are absorbed via additive schema evolution. Tables are partitioned by ingestion date to support retention policies and efficient time-range queries.

#### Silver Entity Pattern

Entity tables store normalized dimension data. Each follows a standard structure:

- `id` — surrogate key (auto-generated).

- `natural_key` — the identifier from the source system (e.g., repository full name, application `sys_id`).
- `source_system` — which tool or platform provided the record.
- `valid_from` / `valid_to` — timestamps for change tracking (Type 2 slowly changing dimension). A `NULL` value in `valid_to` indicates the current version.
- Domain-specific columns — attributes defined by the entity type (e.g., `criticality_tier` for applications, `primary_language` for repositories).

The natural key plus source system combination uniquely identifies a record's origin. The surrogate key provides a stable reference for foreign keys even when source identifiers change.

### Silver Finding Pattern

Finding tables store normalized fact data. Each follows a standard structure:

- `finding_id` — surrogate key.
- `source_finding_id` — the identifier from the source tool.
- `source_tool` — which scanner produced the finding.
- `repository_id` — foreign key to the repository entity table.
- `severity` — normalized to the canonical four-level scale (critical, high, medium, low).
- `status` — normalized lifecycle state (open, confirmed, resolved, false\_positive, wont\_fix).
- `detected_at` / `resolved_at` — UTC timestamps enabling [MTTR](#) calculation.
- `rule_id` — the tool-specific rule or check that triggered the finding.
- Category-specific columns — attributes unique to the finding type (e.g., `file_path` and `line_range` for [SAST](#), `package_name` and `installed_version` for [SCA](#)).

### Relationship Pattern

Relationship tables store many-to-many mappings between entities. Each contains foreign keys to both sides, a source system indicator, and `valid_from`/`valid_to` timestamps to track when the relationship was active. No payload columns are included; the table's sole purpose is to link entities. Examples include application-to-repository, finding-to-[CVE](#), and cross-tool deduplication links.

### Gold Aggregation Pattern

Aggregation tables follow a grain-metric-period structure:

- **Grain columns** define the level of detail: the entity key (application, team, repository) and time period (day, week, month).
- **Metric columns** store computed values: finding counts by severity, [MTTR](#), [SLA](#) compliance percentage, risk score.
- **Period columns** store the time window: `period_start` and `period_end` ([UTC](#)).
- **Refresh metadata**: `computed_at` timestamp and refresh strategy indicator (incremental or full).

This structure enables consistent querying across all gold tables: filter by grain, aggregate over periods, compare across entities.

### 2.2.2 Entity Model

The silver layer instantiates the schema patterns into concrete tables. This subsection specifies the key tables organized by category: entities (dimensions), findings (facts), reference data, and relationships.

#### Entity Tables

[Table 2.1](#) lists the entity tables and their key domain-specific columns. All tables include the standard silver entity pattern columns (`id`, `natural_key`, `source_system`, `valid_from/valid_to`).

#### Finding Tables

Finding tables share the silver finding pattern columns and add category-specific attributes. [Table 2.2](#) summarizes each.

Each finding table references its repository through `repository_id`. Business application context is derived through the relationship tables rather than stored directly on findings, preserving the many-to-many application-to-repository mapping without duplicating finding records.

#### Reference Tables

Reference tables store external intelligence used for enrichment:

**Table 2.1**  
*Silver Entity Tables*

| Table           | Source   | Key Domain Columns                                                                  |
|-----------------|----------|-------------------------------------------------------------------------------------|
| applications    | CMDB     | name, criticality_tier, lifecycle_status, business_unit, compliance_scope           |
| repositories    | SCM      | full_name, primary_language, visibility, default_branch, archived, last_activity_at |
| teams           | CMDB/SCM | name, parent_team_id, contact_email                                                 |
| commits         | SCM      | sha, author, authored_at, message, repository_id                                    |
| pull_requests   | SCM      | number, title, state, author, created_at, merged_at, repository_id                  |
| pipeline_runs   | CI/CD    | pipeline_id, commit_sha, status, started_at, finished_at, repository_id             |
| dependencies    | SCA      | package_name, installed_version, license, ecosystem, repository_id                  |
| branch_policies | SCM      | required_reviewers, require_status_checks, enforce_admins, repository_id            |

**Table 2.2**  
*Silver Finding Tables*

| Table              | Category-Specific Columns                                         |
|--------------------|-------------------------------------------------------------------|
| sast_findings      | file_path, line_start, line_end, cwe_id, language                 |
| sca_findings       | package_name, installed_version, fixed_version, cve_id, ecosystem |
| secret_findings    | secret_type, file_path, commit_sha, validity_status               |
| dast_findings      | url, http_method, parameter, attack_type                          |
| container_findings | image_name, image_tag, layer, cve_id                              |
| iac_findings       | resource_type, resource_name, file_path, policy_id, benchmark     |

- **vulnerabilities** — [CVE](#) records with description, published date, and [CVSS](#) base score from the [NVD](#).
- **epss\_scores** — daily [EPSS](#) probabilities per [CVE](#), enabling time-series analysis of exploitation likelihood.
- **kev\_entries** — [CISA KEV](#) catalog entries indicating confirmed active exploitation, with the date added.

These tables are refreshed on a scheduled basis from their respective external sources. Findings link to them through [CVE](#) identifiers, forming the three-signal enrichment model described in [Section 1.3](#).

## Relationship Tables

Three relationship tables implement the key mappings identified in the domain model:

- **app\_repo\_mapping** — links applications to repositories (many-to-many). This is the most critical relationship for business context attribution.
- **finding\_cve\_mapping** — links findings to [CVE](#) records, enabling vulnerability enrichment.
- **dedup\_links** — links findings identified as duplicates across tools, preserving traceability to each source report while establishing a canonical finding.

### 2.2.3 Aggregation Model

The gold layer computes consumption-ready metrics from silver data using the aggregation pattern. Each gold table targets a specific stakeholder need identified in [Section 1.5.3](#).

## Application Risk Scores

The `app_risk_scores` table computes a composite risk metric per application per period. Grain columns are `application_id` and time period. Metric columns include: open finding counts by severity, weighted risk score (combining severity, [EPSS](#) probability, and application criticality tier), [SLA](#) compliance percentage, and a trend indicator (improving, stable, degrading). This table serves application owners who need a single view of their application's security posture.

## Team Metrics

The `team_metrics` table aggregates remediation performance per team per period. Metrics include: [MTTR](#) by severity, finding closure rate, new vs. resolved finding ratio, and [SLA](#)

breach count. Leadership uses these metrics for cross-team comparisons and resource allocation decisions.

### **Vulnerability Trends**

The `vulnerability_trends` table provides time-series data at configurable grains (daily, weekly, monthly). Metrics include: new findings introduced, findings resolved, net open count, severity distribution shifts, and mean age of open findings. These trends power the longitudinal dashboards that track organizational progress.

### **Coverage Analysis**

The `coverage_analysis` table identifies gaps in security tool coverage. For each repository, it records which tool categories have produced findings or scan records (indicating active scanning) and which have not. Missing coverage is flagged per category: a repository with no [SAST](#) findings and no [SAST](#) pipeline runs likely lacks static analysis integration. This table enables security teams to prioritize tooling rollout.

### **Extension Guide**

Adding a new gold table follows a consistent process: define the grain (which entity, which time period), specify the metrics (what to compute), write a silver-to-gold [LDP](#) transformation, and configure the refresh strategy (incremental or full). The aggregation pattern ensures all gold tables share a common query interface, so dashboards and reporting tools can consume new tables without structural changes.

## **2.3 Connector Framework**

### **2.3.1 Connector Abstraction**

### **2.3.2 Ingestion Patterns**

### **2.3.3 Transformation Patterns**

## **2.4 Analytics and Serving Layer**

### **2.4.1 Aggregation Patterns**

### **2.4.2 ML Workflows**

### **2.4.3 Serving Patterns**

## **2.5 Deployment and Engineering Practices**

### **2.5.1 Deployment Strategy**

### **2.5.2 Project Structure**

### **2.5.3 Testing Strategy**





# **3. Implementation**

## **3.1 Environment and Deployment**

### **3.1.1 Workspace Setup**

### **3.1.2 Silver Schema**

### **3.1.3 Project Structure and CI/CD**

## **3.2 Connectors**

### **3.2.1 ServiceNow**

### **3.2.2 GitHub**

### **3.2.3 GitLab**

### **3.2.4 SonarQube**

### **3.2.5 Semgrep**

### **3.2.6 Dependency-Track**

### **3.2.7 TruffleHog**

### **3.2.8 Vulnerability Enrichment**

## **3.3 Analytics and Serving**

### **3.3.1 Application-Repository Mapping**

### **3.3.2 Application Risk Scoring**

### **3.3.3 Remediation and Compliance Metrics**

### **3.3.4 Vulnerability Trends**

### **3.3.5 Risk Prediction Model**

# **Conclusion**

**Thesis Outcomes and Contributions**

**Limitations**

**Future Work**

# References

- Akamai Technologies. (2024). *Akamai completes acquisition of API security company Noname*. Retrieved April 11, 2026, from <https://www.akamai.com/newsroom/press-release/akamai-completes-acquisition-of-api-security-company-noname> (cit. on p. 25).
- Anderson, R. (2020). *Security engineering: A guide to building dependable distributed systems* (3rd ed.). John Wiley & Sons. (Cit. on pp. 19, 21).
- Aqua Security. (2024). *Trivy documentation*. Retrieved April 11, 2026, from <https://trivy.dev/> (cit. on p. 22).
- Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. *Proceedings of the 11th Annual Conference on Innovative Data Systems Research (CIDR)* (cit. on pp. 27, 35).
- AXELOS. (2019). *ITIL foundation: ITIL 4 edition*. TSO (The Stationery Office). (Cit. on pp. 12, 13, 15).
- Chacon, S., & Straub, B. (2014). *Pro git* (2nd ed.). Apress. (Cit. on p. 17).
- Chess, B., & West, J. (2007). *Secure programming with static analysis*. Addison-Wesley. (Cit. on p. 20).
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387 (cit. on p. 27).
- Cybersecurity and Infrastructure Security Agency. (2024). *Known exploited vulnerabilities catalog*. Retrieved April 11, 2026, from <https://www.cisa.gov/known-exploited-vulnerabilities-catalog> (cit. on p. 19).
- DAMA International. (2017). *DAMA-DMBOK: Data management body of knowledge* (2nd ed.). Technics Publications. (Cit. on pp. 27, 29).
- Databricks. (2025a). *Lakeflow Spark Declarative Pipelines*. Retrieved April 11, 2026, from <https://docs.databricks.com/aws/en/ldp/> (cit. on p. 36).
- Databricks. (2025b). *What is Unity Catalog?* Retrieved April 11, 2026, from <https://docs.databricks.com/aws/en/data-governance/unity-catalog/> (cit. on p. 36).
- Databricks. (2026a). *Databricks enters security market with launch of Lakewatch: New open, agentic SIEM*. Retrieved April 11, 2026, from <https://www.databricks.com/company/newsroom/press-releases/databricks-enters-security-market-launch-lakewatch-new-open-agentic> (cit. on pp. 32, 35).
- Databricks. (2026b). *Databricks Lakebase is now generally available*. Retrieved April 11, 2026, from <https://www.databricks.com/blog/databricks-lakebase-generally-available> (cit. on p. 36).
- DefectDojo. (2024). *DefectDojo documentation*. Retrieved March 6, 2026, from <https://documentation.defectdojo.com/> (cit. on p. 32).
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. IT Revolution Press. (Cit. on p. 17).

- Forum of Incident Response and Security Teams. (2019). *Common vulnerability scoring system v3.1: Specification document*. Retrieved March 6, 2026, from <https://www.first.org/cvss/v3.1/specification-document> (cit. on p. 19).
- Gardner, D., Zumerle, D., & Bhat, M. (2023). *Innovation insight for application security posture management*. Gartner. Retrieved April 11, 2026, from <https://www.gartner.com/en/documents/4326999> (cit. on p. 32).
- Gartner. (2024). *Market share: IT service management, worldwide, 2024*. Retrieved April 11, 2026, from <https://www.servicenow.com/blogs/2025/no-1-6-tech-workflow-segments> (cit. on p. 13).
- GitHub. (2024). *GitHub REST API documentation*. Retrieved April 11, 2026, from <https://docs.github.com/en/rest> (cit. on pp. 17, 20–22).
- GitHub. (2025a). *About GitHub Advanced Security*. Retrieved April 11, 2026, from <https://docs.github.com/en/get-started/learning-about-github/about-github-advanced-security> (cit. on p. 32).
- GitHub. (2025b). *Octoverse 2025: The state of open source*. Retrieved April 11, 2026, from <https://octoverse.github.com/> (cit. on p. 17).
- GitLab. (2024). *GitLab REST API documentation*. Retrieved April 11, 2026, from <https://docs.gitlab.com/ee/api/rest/> (cit. on pp. 17, 20, 22).
- GitLab. (2025). *Application security testing*. Retrieved April 11, 2026, from [https://docs.gitlab.com/user/application\\_security/](https://docs.gitlab.com/user/application_security/) (cit. on p. 32).
- Inmon, W. H. (2005). *Building the data warehouse* (4th ed.). John Wiley & Sons. (Cit. on p. 27).
- Jacobs, J., Romanosky, S., Adjerd, I., & Baker, W. (2021). Improving vulnerability remediation through better exploit prediction. *Journal of Cybersecurity*, 7(1), tyabo15 (cit. on p. 19).
- JetBrains. (2025). *The state of CI/CD in 2025*. Retrieved April 11, 2026, from <https://blog.jetbrains.com/teamcity/2025/10/the-state-of-cicd/> (cit. on p. 18).
- Kim, G., Humble, J., Debois, P., Willis, J., & Forsgren, N. (2021). *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations* (2nd ed.). IT Revolution Press. (Cit. on pp. 17, 19).
- Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). John Wiley & Sons. (Cit. on pp. 27, 31).
- Lee, D., Wentling, T., Haines, S., & Babu, P. (2024). *Delta Lake: The definitive guide: Modern data lakehouse architectures with data lakes*. O'Reilly Media. (Cit. on pp. 27, 29, 35).
- Linux Foundation. (2024). *SPDX specification*. Retrieved March 6, 2026, from <https://spdx.dev/specifications/> (cit. on pp. 19, 21).
- Mack, S. D. (2023). *The DevSecOps playbook: Deliver continuous security at speed*. John Wiley & Sons. (Cit. on p. 19).
- McGraw, G. (2006). *Software security: Building security in*. Addison-Wesley. (Cit. on p. 19).
- Miloslavskaya, N., & Tolstoy, A. (2016). Big data, fast data and data lake concepts. *Procedia Computer Science*, 88, 300–305 (cit. on p. 27).
- MITRE Corporation. (2024a). *CVE program*. Retrieved March 6, 2026, from <https://www.cve.org/> (cit. on p. 19).
- MITRE Corporation. (2024b). *CWE – common weakness enumeration*. Retrieved April 11, 2026, from <https://cwe.mitre.org/> (cit. on pp. 19, 24).

- National Telecommunications and Information Administration. (2021). *The minimum elements for a software bill of materials (SBOM)*. Retrieved March 6, 2026, from [https://www.ntia.gov/sites/default/files/publications/sbom\\_minimum\\_elements\\_report\\_0.pdf](https://www.ntia.gov/sites/default/files/publications/sbom_minimum_elements_report_0.pdf) (cit. on p. 21).
- OASIS Open. (2024). *Static analysis results interchange format (SARIF) version 2.1.0*. Retrieved March 6, 2026, from <https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html> (cit. on p. 19).
- OCSF Project. (2024). *Open cybersecurity schema framework*. Retrieved March 6, 2026, from <https://schema.ocsf.io/> (cit. on pp. 19, 32).
- Ohm, M., Plate, H., Sykosch, A., & Meier, M. (2020). Backstabber’s knife collection: A review of open source software supply chain attacks. *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 23–43 (cit. on p. 21).
- OpenSSF. (2024). *Supply-chain levels for software artifacts (SLSA)*. Retrieved March 6, 2026, from <https://slsa.dev/> (cit. on p. 21).
- OWASP Foundation. (2021). *OWASP top 10:2021*. Retrieved March 6, 2026, from <https://owasp.org/Top10/> (cit. on p. 19).
- OWASP Foundation. (2023). *OWASP top 10 API security risks – 2023*. Retrieved April 11, 2026, from <https://owasp.org/API-Security/editions/2023/en/0x11-t10/> (cit. on p. 25).
- OWASP Foundation. (2024a). *CycloneDX bill of materials standard*. Retrieved March 6, 2026, from <https://cyclonedx.org/> (cit. on pp. 19, 21).
- OWASP Foundation. (2024b). *OWASP Dependency-Track documentation*. Retrieved April 11, 2026, from <https://docs.dependencytrack.org/> (cit. on p. 21).
- OWASP Foundation. (2024c). *OWASP ZAP documentation*. Retrieved April 11, 2026, from <https://www.zaproxy.org/docs/> (cit. on p. 24).
- Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner’s approach* (9th ed.). McGraw-Hill. (Cit. on p. 16).
- Prisma Cloud by Palo Alto Networks. (2024). *Checkov documentation*. Retrieved April 11, 2026, from <https://www.checkov.io/1.Welcome/What%5C%20is%5C%20Checkov.html> (cit. on p. 23).
- Reis, J., & Housley, M. (2022). *Fundamentals of data engineering: Plan and build robust data systems*. O’Reilly Media. (Cit. on p. 28).
- Salt Security. (2024). *Salt Security platform*. Retrieved April 11, 2026, from <https://salt.security/> (cit. on p. 25).
- Scholl, Z. (2024). *Gitleaks: Protect and discover secrets*. Retrieved April 11, 2026, from <https://github.com/gitleaks/gitleaks> (cit. on p. 22).
- Semgrep. (2024). *Semgrep documentation*. Retrieved April 11, 2026, from <https://semgrep.dev/docs/> (cit. on p. 20).
- ServiceNow. (2024a). *CMDB instance API reference*. Retrieved March 7, 2026, from <https://www.servicenow.com/docs/bundle/xanadu-api-reference/page/integrate/inbound-rest/concept/cmdb-instance-api.html> (cit. on pp. 13, 14).

- ServiceNow. (2024b). *Scripted REST APIs*. Retrieved April 11, 2026, from [https://www.servicenow.com/docs/bundle/yokohama-api-reference/page/integrate/custom-web-services/concept/c\\_CustomWebServices.html](https://www.servicenow.com/docs/bundle/yokohama-api-reference/page/integrate/custom-web-services/concept/c_CustomWebServices.html) (cit. on p. 15).
- ServiceNow. (2024c). *Table API reference*. Retrieved March 7, 2026, from [https://www.servicenow.com/docs/bundle/xanadu-api-reference/page/integrate/inbound-rest/concept/c\\_TableAPI.html](https://www.servicenow.com/docs/bundle/xanadu-api-reference/page/integrate/inbound-rest/concept/c_TableAPI.html) (cit. on p. 14).
- Skoulrikari, A. (2024). *Learning git: A hands-on and visual guide to the basics of git*. O'Reilly Media. (Cit. on p. 17).
- Snyk. (2024). *Snyk API documentation*. Retrieved April 11, 2026, from <https://docs.snyk.io/snyk-api> (cit. on p. 21).
- Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson. (Cit. on p. 16).
- SonarSource. (2024). *SonarQube web API documentation*. Retrieved April 11, 2026, from <https://docs.sonarsource.com/sonarqube-server/latest/extension-guide/web-api/> (cit. on p. 20).
- Stack Overflow. (2025). *2025 stack overflow developer survey*. Retrieved April 11, 2026, from <https://survey.stackoverflow.co/2025/> (cit. on pp. 17, 18).
- Stallings, W., & Brown, L. (2024). *Computer security: Principles and practice* (5th ed.). Pearson. (Cit. on pp. 13, 19).
- Strengtholt, P. (2025). *Building medallion architectures: Designing with Delta Lake and Spark*. O'Reilly Media. (Cit. on p. 28).
- Stuttard, D., & Pinto, M. (2011). *The web application hacker's handbook: Finding and exploiting security flaws* (2nd ed.). John Wiley & Sons. (Cit. on p. 23).
- Thalpati, G. A. (2024). *Practical lakehouse architecture: Designing and implementing modern data platforms at scale*. Apress. (Cit. on pp. 27, 28).
- Truffle Security. (2024). *TruffleHog documentation*. Retrieved April 11, 2026, from <https://trufflesecurity.com/trufflehog> (cit. on p. 22).
- Williams, R., & Gonzalez, K. (2021). *3 steps to improve IT service view CMDB data quality*. Retrieved April 11, 2026, from <https://www.gartner.com/en/documents/3994127> (cit. on p. 15).
- Winters, T., Manshreck, T., & Wright, H. (2020). *Software engineering at Google: Lessons learned from programming over time*. O'Reilly Media. (Cit. on p. 17).
- XBOW. (2024). *Autonomous offensive security platform*. Retrieved April 11, 2026, from <https://xbow.com/platform> (cit. on p. 24).

# **Appendices**

# **A. Generative AI Use Disclosure**

## **A.1 Text Content**

## **A.2 Diagrams and Figures**

## **A.3 Source Code**

## **A.4 Requirements Specification**